

Phase 3—The Rest of the Owl

There is no “official” documentation for the requirements of Phase 3, because you won’t need it. The Spec provided at the beginning of the course contains everything needed for Phases 1, 2, and 3. This is just an additional guide covering the main features you will be adding. You may implement them in any order.

Phase 3 Types—Reference Cells

Thus far, all the PSI values you’ve created have corresponded to immutable data. You can operate with lists using `head`, `tail`, and `cons`:

```
psi> (def l (cons 1 (cons 2 (cons 3 ())))))
(1 2 3)
psi> (head l)
1
psi> (tail l)
(2 3)
psi> l
(1 2 3)
```

but there is no way to change a PSI value in-place—note that neither `head` nor `tail` alters the list, `l`. Instead, you create a new PSI value.

This allows us to assume that:

- every value can be copied, and the copies can be treated as identical to the originals—all instances of the number 5 are identical.
- all “sub-values” of composite values (e.g., lists) are uniquely owned by the composite and can safely be deleted when it goes out of scope.

Mutable data breaks those assumptions.

A reference cell—the next type you’ll implement—is a container that holds exactly one PSI value. They are created using the `cell` function, with arity of strictly one:

```
psi> (def e (cell 137))
$cell{137}@0x600001f16880
psi> (def f (cell '(1 2 3)))
$cell{(1 2 3)}@0x600001f14280
psi> (def g (cell +))
$cell{$builtin{+}}@0x600001f18ac0
```

Cells can contain PSI values of any kind, except errors. They are read using the ! (or dereferencing) function and written-to using :=, a mutable assignment function.

```
psi> (! e)
137
psi> (:= e (+ 1 (! e)))
138
psi> e
$cell{138}@0x600001f16880
```

The cell's memory addresses never changes—it will not be the same from run to run; your results will differ from mine—and functions as the “identity” of the cell that allows us to differentiate between two that hold the same value.

```
psi> (def m (cell 193))
$cell{193}@0x600001f16740
psi> (def n (cell 193))
$cell{193}@0x600001f16b00
```

Since they are different cells, a write to m will have no effect on n. Since we desire stability in our definition of equality, we consider cells identical only if they're the same one (“pointer equality.”) Thus, even though their contents are identical, they're different.

```
psi> (= (! m) (! n))
#t
psi> (= m n)
#f
```

This makes sense because, of course, their contents can diverge in the future. After the update below, m holds 193 but n holds 194.

```
psi> (:= n (+ 1 (! n)))
194
psi> (= (! m) (! n))
#f
```

On the other hand, aliasing—two names bound to the same cell—can also occur.

```
psi> (def o m)
$cell{193}@0x600001f16740
psi> (= o m)
#t
```

If these seem “like pointers” to you, then your intuition is good, because they are *exactly* like pointers, except without any support for pointer arithmetic—as their addresses are heap-allocated pointers, no assumptions can be made about where they will be in relation to each other.

In C, argument passing is by-value—for non-pointer data, this means the values themselves are copied when function calls are made; for pointers, it means the references are copied but the pointed-to objects are not. PSI works the same way;

So here’s where it gets tricky. Consider this:

```
psi> (def fives (cons 5 (cons (cell (+ 2 2)) (cons "5" (cons "five" ())))))  
(5 $cell{4}@0x600001f00180 "5" "five")
```

This is almost correct, but PSI didn’t read Orwell and doesn’t know that I intended for 2 + 2 to equal 5. Whoops. I’d like to correct that. This is one of those cases where the term “side effects” is inappropriate—I *want* the stateful effect.

```
psi> (:= (head (tail fives)) (+ 2 3))  
5
```

Has the change propagated to its rightful place?

```
psi> fives  
(5 $cell{5}@0x600001f00180 "5" "five")
```

Yes! But chances are, when you’ve operated with lists, you’ve copied intermediate computations—you’ve written `tail` to create a deep copy of the list, with the leading element removed. That’s a valid approach, but the copying has to stop before reference cells are cloned, because a clone of a reference cell would be one with the same data, but a new address, and they would diverge. Reference cells are *uncopyable*. Instead of copying one, we “up” it—return a pointer, and increment its reference count. Instead of deleting one, we “down” it—decrement its reference count, and free it only if the reference count is zero. This was discussed in more detail in the reference counting module.

Thus, your `pval* pval_copy(pval* pv)` function (your name may vary) cannot *actually* always copy—when `pv` is a reference cell, it instead increments the reference count and returns the same pointer. And your `void pval_delete(pval* pv)` function, when `pv` is a reference cell, cannot demand deletion of the cell but should only decrement the reference count—deletion should only happen if said count is 0, indicating that no other live values rely on it.

This may seem complicated, but it’s worth getting right, because you’ll need it when we get to environments and closures.

And now you can say you have implemented a (very simple, and imperfect, for will not collect cyclic unreachable data structures) garbage collector.

Phase 3 Types—Closures

To make PSI a real programming language, we need the ability to add user-defined functions, like so:

```
psi> (def identity (fn (x) x))
$lambda{(x) x}
psi> (identity 5)
5
psi> (def cube (fn (x) (* x x x)))
$lambda{(x) (* x x x)}
psi> (cube 7)
343
```

Here we introduce a new special form, *fn*, which is the most complicated one you'll encounter. It's what we use to create new functions.

You are free to implement this however you like, but your interior type system will probably want to differentiate between builtin functions (wrappers around function pointers, the function's work done in C) and user-defined functions, since they have different needs. The symbol *** is bound to a builtin function implemented in C, but *cube* exists only in the PSI interpreter—the C code is unaware of its existence.

To be technical, *cube* is a *closure* (or “lambda”)—or function object. It doesn't have a corresponding function pointer, because there is no compiled function for it. Instead, it's an object in memory that PSI recognizes and uses to implement whatever function the closure's creator (the user) intended it to compute.

Lambdas can have fixed arity of any size; examples of 0 and 2 are below.

```
psi> (def const-five (fn () 5))
$lambda{() 5}
psi> (const-five)
5
psi> (def max (fn (x y) (if (> x y) x y)))
$lambda{(x y) (if (> x y) x y)}
psi> (max -13 -4)
-4
```

The behavior above shouldn't be surprising. When `max` is called with `-13` and `-4` as its arguments, it evaluates the body—`(if (> x y) x y)`—as if it were in an environment with `x` bound to `-13` and `y` bound to `-4`.

Lambdas can have variable arity. As discussed in the Spec, one function you can implement in `__start.psi` (an optional file that your interpreter will run before starting up) is `list`, rather than coding it in your C implementation, which takes zero-or-more arguments and returns them as a list:

```
psi> (def list (fn (& xs) xs))
$lambda{(& xs) xs}
psi> (list)
()
psi> (list 1 2 3)
(1 2 3)
```

The `&` symbol is never the name of an argument, but a signifier that the symbol after it will be bound to all the additional arguments, wrapped up in the form of a list. Thus, a function of the form `(fn (x y & more) ...)` requires at least 2 parameters, but can take more.

We want closures, in general, to be stable, which means we want lexical scoping—we want the closure to be protected from later changes to bindings. For example:

```
psi> (def z 1)
1
psi> (def f (fn (x) (+ x z)))
$lambda{(x) (+ x z)}
psi> (f 7)
8
psi> (def z 100)
100
psi> (f 7)
8
psi> f
$lambda{(x) (+ x z)}
```

In other words, we define `f` to compute `(+ x z)` based on a value of `z` that is available at the time: 1. We don't want `f` to change its behavior when `z` is later rebound, which means that `f` has to store an old value of `z`.

Closures can also retain bindings that would otherwise become unreachable (and be deleted) as below:

```
psi> (def g ((fn (x) (fn (y) (+ x y))) 5))
$lambda{(y) (+ x y)}
psi> (g 13)
18
```

We first create an anonymous function `(fn (x) (fn (y) (+ x y)))`—itself a closure, but a short-lived one—and then feed it the value 5, which it binds to its local `x`. This anonymous function returns a different closure, `(fn (y) (+ x y))`, that will outlive it. So, in order to work properly, the outer closure needs to carry a record of what `x` was at the time it was created, which means it needs some way to remember that `x` was 5 when it was spawned, even though no such change was made to the global environment. If this sounds complicated, don't worry—we'll explain how to do it shortly.

A function, when it is “live,” must operate as if its argument values were bound to its argument names (formal parameters.) It must also have access to bindings that existed when it was created. Last but not last—there's a joke, wait for it—there is one more name that a function may need to know—it's own. Functions can be recursive.

```
psi> (def h (fn this (x) (if (= x 0) 1 (* x (this (- x 1))))))
$lambda{(x) (if (= x 0) 1 (* x (this (- x 1))))}@this
psi> (h 5)
120
```

In this case, the function's name precedes its argument list in the `fn` form, and `this` is bound, in the function's execution argument, to the function itself.

It would be more common to name the variable and the function identically, like so:

```
psi> (def factorial (fn factorial (x) (if (= x 0) 1 (* x (factorial (- x 1))))))
```

In order to avoid having to write `(def sym (fn ...))` all the time, we extend the `def`-form by allowing it to have either two or three subforms.

- one with two subforms—e.g., `(def a 5)`—preserves its prior semantics.
- one with three subforms, if the second is a list of symbols, defines a function:
 - `(def sym arglist body)` is equivalent to `(def sym (fn sym arglist body))`, always using the named (3-argument) variant of the `fn`-form.

Functions thus defined are always “named” even if they are not recursive.

```
psi> (def fct (x) (if (= x 0) 1 (* x (fct (- x 1)))))
$lambda{(x) (if (= x 0) 1 (* x (fct (- x 1))}}@fct
psi> (fct 7)
```

5040

```
psi> (def even (x) (= (% x 2) 0))  
$Lambda{(x) (= (% x 2) 0)}@even
```

In order to understand functions and how they operate, we thus need to understand environments.

How To Do Environments

This is a C course, not a Lisp course. You do not need to puzzle over how environments work—I will tell you what you need to do.

In Phase 1, there was no environment—a form like `(+ (* 4 5) 6)` always evaluated to 26, having no dependencies.

In Phase 2, we had a single global environment—we could `(def x 7)` to create a binding that would enable `(+ 5 x)` to evaluate to 12 later on. Thus, evaluation (the “eval” in “read-eval-print”) is parameterized by an environment, but we only had to worry about the single global one.

In Phase 3, we find ourselves in need of more—for example, if *y* and *z* are parameter names, then the body of a function `(+ y z)` operates in a different environment than a toplevel `(+ y z)`.

An environment is a data structure with:

- an optional *parent environment*. Global environments, and only they, are parentless.
- a list of *bindings* of names (symbols) to PSI values.

You are free to implement environments however you like. A regular list is slower than a hash table, but simple to implement, and fast enough for our purposes.

When the REPL starts, the global environment is the active environment and it contains only the builtin functions, so we could write that environment like so:

```
e0 = {+: BuiltinFn(...), -: BuiltinFn(...), ... } with no parent.
```

The `def` special form creates new bindings in the environment where it executes. Thus, after

```
psi> (def a 5)  
5  
psi> (def b (+ a 7))  
12
```

... the state of our global environment is:

```
e0 = {a: Number(5), b: Number(12), +: BuiltinFn(...), ... }
```

So that:

```
psi> (+ a b)
17
```

The `let` special form creates a local environment with a new binding. For example,

```
psi> (let c 37 (+ a c))
42
```

Here, we create a local environment:

```
e1 = {c : Number(37)} with parent e0.
e0 = {a : Number(5), Number(12), ... } with no parent.
```

In `e1`, we evaluate `(+ a c)` to 42; this `e1` is discarded once the `let`-form exits. In cases of conflict, the more local binding wins. For example,

```
psi> (let d (+ a a) (let a (+ d d) (+ a 1)))
21
```

This creates the environment:

```
e2 = {a : Number(20)} with parent e1.
e1 = {d : Number(10)} with parent e0.
e0 = {a : Number(5), ...} with no parent.
```

Since we are in `e2` when the final `(+ a 1)` is evaluated, we return 21—the `e2` binding doesn't delete the global one, but it shadows it.

Closures use local environments to operate. Consider the following sequence, and recall that `b : Number(12)` in the environment above.

```
psi> (def f (fn (a) (+ a b)))
$lambda{(a) (+ a b)}
psi> (f 26)
38
psi> a
5
```


When the body of `f`, `(+ a b)` is being evaluated, `a` is bound to `Number(26)`. This is achieved using a local environment that exists during the function call, and that is discarded after it. What should the local environment's parent be? Not the calling environment, but the closure's creation-time environment. This is why function objects are called closures—they “close over” the environment that exists when they are created.

So, here are the rules for environment:

- at **closure creation time**, create a full deep copy, down to the global environment, of the environment in which the fn-form is evaluated. This is a “frozen” form of the function's environment that will never change.
- at **function call time**, create an environment `e1` whose parent is the function's stored/frozen environment and that includes bindings for each parameter—and, if it's a named function, one for itself.
 - for example, if applying `(fn (x y) (+ x y))` to arguments `6 12`, then `e1` includes `{x : 6, y : 12}`.
- at **function return time**, transfer back to the environment that existed prior to the function call.

It may take you some time to understand this. Read it early; read it often. Ask questions on Ed.

The closure struct I use—your implementation may differ—is below:

```
typedef struct closure {
    bool is_macro;
    int ref_count;
    pval* fn_name;
    pval* arglist;
    pval* body;
    env* env;
} closure;
```

The `fn_name` may be `NULL` if it's not a named function, as non-recursive functions don't need names. The `arglist` is always a list of symbols; anything else would be a type error that is rejected before the closure is made. The `body` is an arbitrary PSI value. We store `env` because we need to preserve the closure's creation-time environment.

Notice that closures also have a reference count. Like the mutable reference cells above, closures are uncopyable. Rather than copy a closure, you “up” it by incrementing its reference count while “returning” the original pointer; rather than delete it, you “down” it by decrementing the reference count and only actually deleting on zero. **This is not optional.** Since closures retain creation-time environments, and environments bind symbols to values, which may be closures, an implementation that copied closure objects would end up doing an exponential amount of cloning. Thus, while we have to copy environments—this is ugly, but necessary to make closures work—we should never copy closure objects.

Once you've implemented closures properly, you've done the hardest part of Phase 3. It's all downhill from here.

The eval Function

The `eval` function exposes one step of PSI interpretation directly to the user, like so:

```
psi> (eval '(+ 3 5))  
8
```

Also, `eval` is the only builtin with environmental dependencies. It takes a PSI list and performs evaluation on it within the given environment.

```
psi> (eval 'y)  
$error{(unbound y)}  
psi> ((fn (y) (eval 'y)) 6)  
6  
psi> (def y 7)  
7  
psi> (eval 'y)  
7
```

The other builtins are—by design—environment-independent, like mathematical functions. But `eval`, which answers the “What does this mean?” question, has an intrinsic environmental dependency.

Reading with read and parse

Although you've already implemented reading—that is, turning strings into PSI S-expressions—you'll expose those capabilities with `read` and `parse`, as implemented in the Spec. Don't worry about `parse` too much—it's more complicated, and it's not used in the unit tests—but you should get `read` to a point where it passes all the test cases.

```
psi> (read "(+ 3 4)")  
(+ 3 4)  
psi> (read "\"cat\"")  
"cat"  
psi> (read "")  
$error{(value-error read "")}
```

Printing with print and str

We've discussed `read` and `eval`—it should not surprise you that we're discussing `print`. First, though, we should describe `str`, which converts a value to its readable/printable string form:

```
psi> (str 5)
"5"
psi> (str "cat")
"\cat\"
psi> (str (cell 5))
"$cell{5}@0x600002418140"
```

Of course, some values (those prefixed with `$`) are not readable, but useful information should be conveyed.

One user of this functionality is `print`, which prints its arguments in space-separated form:

```
psi> (print 1 "1" 111)
1 "1" 111
#t
```

This is equivalent to `(output (str 1) " " (str "1") " " (str 111) "\n")`. As with `output`, the return value is `#t`. The other stuff is direct console I/O.

An error Function

The `error` function takes a string and creates an error value. Since errors propagate through functions, there is no distinction in PSI between returning an error value and throwing it.

```
psi> (error "I've got le bad news.")
$error{"I've got le bad news."}
```

The `do` Special Form

We use `do` when we want to execute a sequence of forms, for stateful effects, but don't care to do anything with their return values.

For example, here is a function that prints the first three powers of its argument, then returns a meaningless `#t`.

```
psi> (def f (n) (do (print n) (print (* n n)) (print (* n n n)) #t))
f@(\ n . (do (print n) (print (* n n)) (print (* n n n)) #t))
psi> (f 7)
7
49
```

343

`#t`

The semantics of `(do <form-1> <form-2> ... <form-N>)` are:

- evaluate the forms in order, from left to right.
- if an error occurs, return the first one and do not execute any subsequent forms.
- otherwise, return the results of evaluating the *last* form.

The loop Special Form

The `loop` special form takes one subform and evaluates it over and over until it returns `#f` or throws an error. If it exits normally, it evaluates to `#t` to indicate successful execution. If it throws an error, the error is thrown.

For example:

```
psi> (loop #f)
#t
psi> (loop (/ 1 0))
$error{division-by-zero}
psi> (loop #t)
```

The last of these never returns. As you see, this form isn't interesting at all when it's given a constant. However, it can be used with mutable reference cells to implement the familiar loop structures that high-level languages have. For example:

```
psi> (def acc (cell 0))
$cell{0}@0x6000028ccb00
psi> (def ctr (cell 0))
$cell{0}@0x6000028c8240
psi> (loop (if (<= (! ctr) 144)
              (do (:= acc (+ (! acc) (* (! ctr) (! ctr))))
                  (:= ctr (+ 1 (! ctr)))
                  #t)
              #f))
#t
psi> (! acc)
1005720
```

We set an accumulator and a loop counter to 0. Then, using the `loop` special form, we run a loop that first checks whether our counter exceeds 144—if it does, we exit the loop—and, if it doesn't, adds the square of the counter's value to the accumulator before incrementing the counter. An equivalent snippet of C code would be:

```
int acc = 0;
int ctr = 0;
while (ctr <= 144) {
    acc += ctr * ctr;
    ctr += 1;
}
```

The try special form

In PSI, errors propagate—that is, if an error occurs anywhere in a computation, the whole computation returns that error, regardless of the function calls or call stack. What if we want to implement more graceful error handling—capture errors, like exceptions? Since errors propagate through functions, per function semantics, we need a special form for this.

The behavior of `(try <form>)` is to evaluate that form and return a two-element list. The first element will be a boolean indicating whether the evaluation returned normally (i.e., a non-error) or abnormally. The second will be, in the case of normal return, the value itself, and in the case of an error, the string inside it. Thus,

```
psi> (try (+ 4 5))
(#t 9)
psi> (try (/ 1 0))
(#f division-by-zero)
```

You can use this to implement exception handling strategies (function definition reformatted for clarity):

```
psi> (def divide-100-by-input ()
      (let result (try (/ 100 (read (input)))))
      (if (head result) (head (tail result))
          "You divided by zero, you idiot.")))
divide-100-by-input@(\ ... )
psi> (divide-100-by-input)
20
5
psi> (divide-100-by-input)
0
"You divided by zero, you idiot."
```

Three Features, Two Routes—and, or, and let

The next part of the assignment has two routes. You will implement, either as hardcoded special forms in C, or by implementing macros, support for `and`, `or`, and `let`.

The first two of these are short-circuiting logical operators; like `if`-forms, they stop evaluating as soon as the truth or falsity of the expression is known. To evaluate `(or <expr-1> ... <expr-N>)`:

- if there are no subexpressions (i.e., $N = 0$) return `#f`.
- evaluate `<expr-1>` and, if the result is true—that is, anything not `#f`—return it; the rest of the expressions are never evaluated. If the result is `#f`, then evaluate `<expr-2>` and apply the same reasoning, going on until you either find (and return) a truthy value or exhaust the list—in this case, return `#f`.

This can *almost* be expressed recursively like so:

- base case: `(or) → #f`
- general case: `(or <expr-1> <expr-2> ...)` → `(if <expr-1> <expr-1> (or <expr-2> ...))`

Almost. The `if`-expansion above **isn't quite right**. It does compute and return the correct result, but if evaluating `<expr-1>` has stateful effects, they will happen twice. We'll discuss this later. If you implement `or` as a builtin special form, you won't have to worry about this—simply evaluate left-to-right, returning the first truthy value with no further evaluation.

To evaluate `(and <expr-1> ... <expr-N>)`:

- if there are no subexpressions (i.e., $N = 0$) return `#t`.
- evaluate `<expr-1>` and, if the result is `#f`, return it; otherwise, apply the same reasoning rightward (e.g., to `<expr-2>`, ..., `<expr-N>`) until you get `#f` or exhaust the list, in which case you return the results of the last (rightmost) expression.

This *can* be expressed recursively:

- base case 0: `(and) → #t`
- base case 1: `(and <expr-1>) → <expr-1>`
- general case: `(and <expr-1> <expr-2> ...)` → `(if <expr-1> (and <expr-2> ...) #f)`

As before, you are free to implement this short-circuiting `and` as a special form—but you can also use macros.

Finally, `let` is a form whose purpose is to create local variables, as most complex functions will require. There will always be an odd number of subforms. The name forms must be symbols.

The behavior of `(let <name-1> <expr-1> <name-2> <expr-2> ... <body>)` is:

- evaluate `<expr-1>` and create a new local environment (atop the existing one) that binds its result to `<name-1>`;

- do the same for <expr-2> into <name-2> and so on... note that <name-1> is available in <expr-2>, that <name-1> and <name-2> are available to <expr-3>... and that all names are available in <body>;
- in the generated local scope, evaluate <body> and return the result...
- and return to the calling environment (discarding the local frames) once done.

For an example, reformatted for clarity:

```
psi> (let a 5
      b (+ a 2)
      c (* a b)
      (list a b c))
(5 7 35)
```

This seems like it would be a messy thing to implement, but if you've implemented closures properly, you already have the machinery for doing so. We can express `let` recursively:

- base case 3: `(let <name> <expr> <body>) → ((fn (<name>) <body>) <expr>)`
- general case: `(let <name> <expr> <form>*) → ((fn (<name>) (let <form>*)) <expr>)`

For example, the form `(let a 5 b (* a a) (list a b))` expands to:

```
( (fn (a) (let b (* a a) (list a b))) 5 ) →
( (fn (a) ((fn (b) (list a b)) (* a a))) 5 )
```

which is then evaluated identically to `(list 5 25)`. We see immediately the advantage of the `let`-form—it's much more legible.

A working `let` form also enables us to understand correct semantics of `or`, which can be stated thus:

- base case: `(or) → #f`
- general case: `(or <expr-1> <expr-2> ...) → (let <name> <expr-1> (if <name> <name> (or <expr-2> ...)))`

which solves our problem by ensuring that <expr-1> is only evaluated once, with the caveat that <name> must be a symbol that appears nowhere in <expr-2> ... <expr-N>.

The Macro Way

You are allowed to implement `and`, `or`, and `let` as hard-coded special forms, in your C code, but there's another way to do this, which is to implement *macros*, which are functions that generate code.

The provided standard library ("`__start.psi`") defines these three forms as macros:

```
(macro and (& x)
  (if (= x ()) #t
    (if (= (tail x) ())
      (head x)
      (list 'if (head x)
              (cons 'and (tail x))
              #f))))))

(macro let (& x)
  (if (= (% (length x) 2) 0)
    (error (list 'arity-error 'let 'odd (length x)))
    (if (= (tail x) ()) (head x)
      (if (= (type (head x)) 'symbol)
        (list (list 'fn (list (head x))
                            (cons 'let (tail (tail x))))
              (head (tail x)))
        (error (list 'type-error 'let 1 'symbol (head x)))))))

(macro or (& x)
  (if (= x ()) #f
    (let y (new-symbol)
      (list
        (list 'fn (list y)
              (list 'if y y (cons 'or (tail x))))
        (head x)))))
```

If your implementation does not support macros, you will want to remove these lines and, instead, implement those special forms.

If you take the macro route, you'll want to implement `new-symbol`, which generates new symbols that are guaranteed to exist nowhere else in the program, like so:

```
psi> (new-symbol)
g$24
psi> (new-symbol)
g$25
psi> (new-symbol)
```


g\$26

The `$` is the reader-reject character—it is allowed in symbols, but the reader knows to reject symbols from user strings that contain it. Thus, an attempt to maliciously collide with `new-symbol` would fail:

```
psi> (def g$27 1)
$error{(invalid-token "$")}
psi> (read "g$27")
$error{(value-error read "g$27")}
```

Only symbols returned by `new-symbol` are allowed to contain this character.

Macros are closures that, instead of evaluating their arguments like functions, operate on the unevaluated forms—that is, operate on code—to generate new code that will then be evaluated.

So, for example, `(and 5 6)` ends up returning:

```
(list 'if 5 (cons 'and '(6)) #f) -> (if 5 (and 6) #f)
```

which is then evaluated, because the condition is true, to:

```
(and 6) -> (if 6 (and) #f)
```

which is then evaluated to:

```
(and) -> #t.
```

To see why we need `new-symbol`, let's implement `or`. As discussed above, we need a construction that ensures that the first argument, if true, is still only evaluated once. We might be tempted to write `or` as:

```
(macro bad-or (& x)
  (if (= x ()) #f
      (list 'let 'zyx (head x)
            (list 'if 'zyx 'zyx (cons 'or (tail x))))))
```

And this would *almost* work.

```
psi> (bad-or)
#f
psi> (bad-or (= 3 3) (/ 1 0))
#t
```

The problem is that, if we ever depend on `zyx` having a value, it breaks down:

```
psi> ((fn (zyx) (or #f (+ 1 zyx))) 7)
8
psi> ((fn (zyx) (bad-or #f (+ 1 zyx))) 7)
$error{(type-error + 2 number #f)}
```

The `bad-or` macro is unhygienic—it assumes “no one is using” the variable name `zyx`, but if anyone is, it shadows the existing definition and causes unexpected behaviors. With `new-symbol`, this can be made hygienic:

```
(macro better-or (& x)
  (if (= x ()) #f
      (let ns (new-symbol)
        (list 'let ns (head x)
              (list 'if ns ns (cons 'or (tail x)))))))
```

Macros, of the Lisp variety, allow users to create new language constructs that generate code for the user. They are, however, difficult to create and even harder to debug. The good news is that you do not have to create any macros for this project—again, this is a C course, not a Lisp course—and 95% of the complexity is around how to use macros, not how to support them. If you have closures, you basically have everything you need to support them but, as already said, you don’t have to do so. You can implement `and`, `or`, and `let` by hand—if you do so, remove the macro definitions from the provided `__start.psi`.

The special form `macro` creates a macro. Like `def` when defining a function, it takes a name (symbol) and an argument list (list of symbols) as well as a body (PSI form). The difference is that the `is_macro` flag in the closure is set.

```
psi> (def identity (x) x)
$lambda{(x) x}@identity
psi> (macro m-identity (x) x)
$macro{(x) x}@m-identity
psi> (identity 5)
5
psi> (m-identity 5)
5
```

They are deceptively similar so far, but they’ll diverge soon. This example uses the `range` and `length` functions defined in the standard library:

```
psi> (range 0 10)
(0 1 2 3 4 5 6 7 8 9)
```

```

psi> (length (range 0 10))
10
psi> (macro m-length (xs) (length xs))
$macro{(xs) (length xs)}@m-length
psi> (m-length (range 0 10))
3

```

Macros, like special forms, operate on code—they operate on forms. Therefore, what `m-length` does is *not* compute apply `length` to the computed value of `(range 0 10)` but, instead, apply it to the form itself—the form `(range 0 10)` is a list of length 3, so it returns 3.

Application of the macro closure to the forms is called *macroexpansion*—a translation step that turns code into other code—and, after macroexpansion, the result is evaluated.

As I said, the concept here is difficult, but supporting macros, in your C code, won't be that bad. To do it, you simply need to:

- Implement and expose the `new-symbol` function, on which macros depend. Aside from its necessary stateful behavior (the auto-incrementation) it is just an ordinary function.
- Implement an `is_macro` flag on closures.
- When applying a closure, check the `is_macro` flag. If it is not set, apply the closure as usual. If it is set, do not evaluate the subforms, but instead apply the macro body to the unevaluated forms—they are code, not values.
- Finally, in the macro case, evaluate the generated code.

The type Function

We are now truly down to the odds-and-ends, so to speak. You will implement a type function. It takes one argument, and returns the type of it as a symbol.

```

psi> (type 1)
number
psi> (type 'a)
symbol
psi> (type "a")
string
psi> (type ())
list
psi> (type '(1 2 3))
list
psi> (type (fn (x) x))
function
psi> (type +)
function

```

```
psi> (type #t)
bool
psi> (type (cell 2))
cell
```

The % (Modulus) Function

You will expose as % the modulus function. Unlike C's version, you will always return the positive modulus, e.g.:

```
psi> (% -1 100)
99
```

It is an error if the modulus is zero.

```
psi> (% 6 0)
ERROR (% : division by zero)
```

Optional: Graceful Input Handling

In Phase 1 and Phase 2, you took input one line at a time, parsed it as an S-expression, and converted it into a PSI value. If it was a bad parse, you rejected it, like so:

```
psi> (def a 13
ERROR (unclosed list)
```

You can keep your interpreter for Phase 3 the same way, or you can improve it by having it:

- await further input in the case of unclosed parentheses or quotation marks,
- evaluate multiple expressions on the same line.

```
psi> (+ (* 4 5)
        (* 6 7))
62
psi> (* 8 9) (* 10 11)
72
110
psi> "cats
      dogs"
"cats\ndogs"
```

The first is a case of a list being expressed over multiple lines; the second has two expressions on the same line, and the REPL prints the values of both. The third is a multiline string—the newline is *not* thrown away, because it is relevant.

You don't have to do this part, but you probably want to, because it makes your REPL nicer to use, and you'll also need to handle multiline expressions, and multiple expressions, for...

The load Function

The `load` function takes a single argument, a string, that corresponds to the name of a file containing PSI code. You can assume, for the purpose of this assignment, that the file is no larger than 1,048,576 bytes (1 MB) and that it will contain correct code—although you should handle erroneous code gracefully, you'll only be graded on correct files.

The function will read and evaluate each of the expressions—in order—as if they were input at the REPL; however, it will not print unless explicitly asked to do so via the `print` function. The return of `load` is the number of expressions parsed and evaluated.

Therefore, if you have a file called `"ex1.psi"` with the following contents:

```
---- ex1.psi ----
(output "loading\n")

(def a 15)

(def b 37)

(* a b)

(print (+ a b))

(output "done\n")
---- END ----
```

You will see the following results:

```
psi> (load "ex1.psi")
loading
52
done
6
```

The 52 was never returned but is a state effect of printing `(+ a b)`. The 6, which was returned, is the number of expressions evaluated in the file.

The bindings will also become available in the REPL after loading:

```
psi> a
15
psi> b
37
```

Finally, you'll implement batch mode. When you run your C process with a command-line argument, you will load that file—that is, read it and evaluate (execute) all of its code. In batch mode, I/O (such as that performed by input, output, print) is done, but the return value of the load is discarded. Like so:

```
$ ./psi test-basic.psi
287 tests passed.
0 tests failed.
```